TPO E-Sports Scrims Matchmaking - Diagrama de Clases Actualizado

domain

strategy

«Strategy» MatchmakingStrategy
- + seleccionar(candidatos: List<Usuario>, scrim: Scrim): List<Usuario>
- + getNombreEstrategia(): String

«ConcreteStrategy» ByHistoryStrategy
- + ByHistoryStrategy()
- + seleccionar(candidatos: List<Usuario>, scrim: Scrim): List<Usuario>
- + getNombreEstrategia(): String

«ConcreteStrategy» ByLatencyStrategy
- - latenciasPorRegion: Map<String, Map<String, Integer>>
- + ByLatencyStrategy()
- + seleccionar(candidatos: List<Usuario>, scrim: Scrim): List<Usuario>
- + getNombreEstrategia(): String
- - calcularLatencia(regionUsuario: String, regionScrim: String): int

«ConcreteStrategy» ByMMRStrategy
- - rangoValores: Map<String, Integer>
- + ByMMRStrategy()
- + seleccionar(candidatos: List<Usuario>, scrim: Scrim): List<Usuario>
- + getNombreEstrategia(): String

moderacion

«ChainOfResponsibility» ReportProcessor
- + procesarReporte(reporte: ReporteConducta): void
- + setSiguiente(siguiente: ReportProcessor): void

«ConcreteHandler» AutomaticProcessor
- - siguiente: ReportProcessor
- + AutomaticProcessor()
- + procesarReporte(reporte: ReporteConducta): void
- + setSiguiente(siguiente: ReportProcessor): void

1 deriva reporte a 0..1

«ConcreteHandler» BotProcessor
- - siguiente: ReportProcessor
- + BotProcessor()
- + procesarReporte(reporte: ReporteConducta): void
- + setSiguiente(siguiente: ReportProcessor): void

1 escala reporte a 0..1

«ConcreteHandler» HumanModeratorProcessor
- - siguiente: ReportProcessor
- + HumanModeratorProcessor()
- + procesarReporte(reporte: ReporteConducta): void
- + setSiguiente(siguiente: ReportProcessor): void

state

«State» ScrimState
- + postular(scrim: Scrim, usuario: Usuario, rolDeseado: String): void
- + confirmar(scrim: Scrim, usuario: Usuario): void
- + iniciar(scrim: Scrim): void
- + finalizar(scrim: Scrim): void
- + cancelar(scrim: Scrim): void
- + getNombre(): String

«ConcreteState» CanceladoState
- + postular(scrim: Scrim, usuario: Usuario, rolDeseado: String): void
- + confirmar(scrim: Scrim, usuario: Usuario): void
- + iniciar(scrim: Scrim): void
- + finalizar(scrim: Scrim): void
- + cancelar(scrim: Scrim): void
- + getNombre(): String

«ConcreteState» FinalizadoState
- + postular(scrim: Scrim, usuario: Usuario, rolDeseado: String): void
- + confirmar(scrim: Scrim, usuario: Usuario): void
- + iniciar(scrim: Scrim): void
- + finalizar(scrim: Scrim): void
- + cancelar(scrim: Scrim): void
- + getNombre(): String

«ConcreteState» EnJuegoState
- + postular(scrim: Scrim, usuario: Usuario, rolDeseado: String): void
- + confirmar(scrim: Scrim, usuario: Usuario): void
- + iniciar(scrim: Scrim): void
- + finalizar(scrim: Scrim): void
- + cancelar(scrim: Scrim): void
- + getNombre(): String

«ConcreteState» ConfirmadoState
- + postular(scrim: Scrim, usuario: Usuario, rolDeseado: String): void
- + confirmar(scrim: Scrim, usuario: Usuario): void
- + iniciar(scrim: Scrim): void
- + finalizar(scrim: Scrim): void
- + cancelar(scrim: Scrim): void
- + getNombre(): String

«ConcreteState» LobbyArmadoState
- + postular(scrim: Scrim, usuario: Usuario, rolDeseado: String): void
- + confirmar(scrim: Scrim, usuario: Usuario): void
- + iniciar(scrim: Scrim): void
- + finalizar(scrim: Scrim): void
- + cancelar(scrim: Scrim): void
- + getNombre(): String

«ConcreteState» BuscandoJugadoresState
- + postular(scrim: Scrim, usuario: Usuario, rolDeseado: String): void
- + confirmar(scrim: Scrim, usuario: Usuario): void
- + iniciar(scrim: Scrim): void
- + finalizar(scrim: Scrim): void
- + cancelar(scrim: Scrim): void
- + getNombre(): String

command

«Command» ScrimCommand
- + execute(scrim: Scrim): void
- + undo(scrim: Scrim): void
- + getDescription(): String

«Invoker» CommandInvoker
- - scrim: Scrim
- - comandosEjecutados: Stack<ScrimCommand>
- - comandosDeshechos: Stack<ScrimCommand>
- + CommandInvoker(scrim: Scrim)
- + ejecutar(comando: ScrimCommand): void
- + deshacer(): void
- + rehacer(): void

1 almacena historial de 0..*

«ConcreteCommand» InvitarJugadorCommand
- - usuarioInvitado: Usuario
- - rolSugerido: String
- + InvitarJugadorCommand(usuarioInvitado: Usuario, rolSugerido: String)
- + execute(scrim: Scrim): void
- + undo(scrim: Scrim): void
- + getDescription(): String

«ConcreteCommand» AsignarRolCommand
- - usuario: Usuario
- - nuevoRol: String
- - rolAnterior: String
- + AsignarRolCommand(usuario: Usuario, nuevoRol: String)
- + execute(scrim: Scrim): void
- + undo(scrim: Scrim): void
- + getDescription(): String

«ConcreteCommand» SwapJugadoresCommand
- - usuario1: Usuario
- - usuario2: Usuario
- + SwapJugadoresCommand(usuario1: Usuario, usuario2: Usuario)
- + execute(scrim: Scrim): void
- + undo(scrim: Scrim): void
- + getDescription(): String

delega comportamiento en
invita a
asigna rol a
intercambia entre

model

ReporteConducta
- - id: UUID
- - scrim: Scrim
- - reportador: Usuario
- - reportado: Usuario
- - motivo: String
- - descripcion: String
- - estado: EstadoReporte
- - sancion: Sancion
- - fechaCreacion: Date
- - fechaResolucion: Date
- - notas: String
- - procesadorResolvio: String
- + ReporteConducta(scrim: Scrim, reportador: Usuario, reportado: Usuario, motivo: String, descripcion: String)
- + esReportePendiente(): boolean
- + marcarResuelto(sancion: Sancion, notas: String, procesador: String): void
- + getReportado(): Usuario
- + getReportador(): Usuario
- + getMotivo(): String
- + getDescripcion(): String
- + getEstado(): EstadoReporte
- + getSancion(): Sancion
- + getNotas(): String

ejecuta comandos sobre
tiene estado 1..1

EstadoReporte
- - nombre: String
- + EstadoReporte(nombre: String)
- + getNombre(): String
- + esPendiente(): boolean
- + esEnRevision(): boolean
- + esResuelto(): boolean
- + esDescartado(): boolean

aplica sanción 1..1 0..1
se origina en 1..1

Sancion
- - nombre: String
- - descripcion: String
- - duracionDias: int
- + Sancion(nombre: String, descripcion: String, duracionDias: int)
- + getNombre(): String
- + getDescripcion(): String
- + getDuracionDias(): int
- + requiereCooldown(): boolean

Confirmacion
- - id: UUID
- - usuario: Usuario
- - scrim: Scrim
- - confirmado: boolean
- - fechaConfirmacion: Date
- + Confirmacion(usuario: Usuario, scrim: Scrim)
- + confirmar(): void
- + isConfirmado(): boolean
- + getUsuario(): Usuario
- + getScrim(): Scrim
- + getFechaConfirmacion(): Date

1 confirma a 1
registra 0..*
reporta a 1
es emitido por 1

Usuario
- - id: UUID
- - username: String
- - email: String
- - passwordHash: String
- - region: String
- - rangoPorJuego: Map<String, String>
- - rolesPreferidos: List<String>
- - preferencias: PreferenciasUsuario
- - strikes: int
- - cooldownHasta: Date
- + Usuario(username: String, email: String, passwordHash: String, region: String)
- + agregarRangoJuego(juego: String, rango: String): void
- + getRangoParaJuego(juego: String): String
- + agregarRolPreferido(rol: String): void
- + estaBajoSancion(): boolean
- + aplicarStrike(): void
- + getId(): UUID
- + getUsername(): String
- + getEmail(): String
- + getRegion(): String
- + getRolesPreferidos(): List<String>
- + getStrikes(): int
- + getPreferencias(): PreferenciasUsuario

posee 1..1 1..1

PreferenciasUsuario
- - juegosPreferidos: List<String>
- - latenciaMaxima: int
- - horariosDisponibles: List<String>
- - recibirNotificacionesPush: boolean
- - recibirNotificacionesEmail: boolean
- - recibirNotificacionesDiscord: boolean
- + PreferenciasUsuario()
- + getJuegosPreferidos(): List<String>
- + getHorariosDisponibles(): List<String>
- + getLatenciaMaxima(): int
- + isRecibirNotificacionesPush(): boolean
- + isRecibirNotificacionesEmail(): boolean
- + isRecibirNotificacionesDiscord(): boolean
- + setRecibirNotificacionesPush(valor: boolean): void
- + setRecibirNotificacionesEmail(valor: boolean): void
- + setRecibirNotificacionesDiscord(valor: boolean): void

Equipo
- - nombre: String
- - jugadores: List<Usuario>
- - rolesAsignados: Map<Usuario, String>
- + Equipo(nombre: String)
- + agregarJugador(usuario: Usuario): void
- + removerJugador(usuario: Usuario): void
- + asignarRol(usuario: Usuario, rol: String): void
- + getRolDeJugador(usuario: Usuario): String
- + getJugadores(): List<Usuario>
- + getNombre(): String

agrupa jugadores 1..1 0..*
asigna roles 1..1 0..*
registra rendimiento 1..1 0..*

Scrim
- - id: UUID
- - creador: Usuario
- - juego: String
- - formato: String
- - cuposTotales: int
- - rolesPorLado: Map<String, Integer>
- - region: String
- - rangoMin: String
- - rangoMax: String
- - latenciaMax: int
- - fechaHora: LocalDateTime
- - duracionEstimada: int
- - modalidad: String
- - estado: ScrimState
- - postulaciones: List<Postulacion>
- - confirmaciones: List<Confirmacion>
- - equipoA: Equipo
- - equipoB: Equipo
- - estadistica: Estadistica
- + Scrim(creador: Usuario, juego: String, formato: String, region: String)
- + postular(usuario: Usuario, rolDeseado: String): void
- + confirmar(usuario: Usuario): void
- + iniciar(): void
- + finalizar(): void
- + cancelar(): void
- + setEstado(estado: ScrimState): void
- + ejecutarMatchmaking(usuariosSeleccionados: List<Usuario>): void
- + getId(): UUID
- + getCreador(): Usuario
- + getJuego(): String
- + getFormato(): String
- + getCuposTotales(): int
- + getRegion(): String
- + getRangoMin(): String
- + getRangoMax(): String
- + getLatenciaMax(): int
- + getEstado(): ScrimState
- + getPostulaciones(): List<Postulacion>
- + getConfirmaciones(): List<Confirmacion>
- + getEquipoA(): Equipo
- + getEquipoB(): Equipo
- + getEstadistica(): Estadistica

organiza 1..1 2
fue creado por 1..1 1
registra resultado en 1..1 0..1

Estadistica
- - id: UUID
- - scrim: Scrim
- - estadisticasPorJugador: Map<Usuario, EstadisticaJugador>
- - equipoGanador: String
- - observaciones: String
- + Estadistica(scrim: Scrim)
- + agregarEstadisticaJugador(usuario: Usuario, estadistica: EstadisticaJugador): void
- + getEstadisticaJugador(usuario: Usuario): EstadisticaJugador
- + getEquipoGanador(): String
- + setEquipoGanador(equipoGanador: String): void
- + getObservaciones(): String
- + setObservaciones(observaciones: String): void

compone resultados por jugador 1..1 0..*

EstadisticaJugador
- - kills: int
- - deaths: int
- - assists: int
- - mvp: boolean
- - rating: int
- + EstadisticaJugador(kills: int, deaths: int, assists: int)
- + getKDA(): double
- + getKills(): int
- + getDeaths(): int
- + getAssists(): int
- + isMvp(): boolean
- + setMvp(mvp: boolean): void
- + getRating(): int
- + setRating(rating: int): void

pertenece a 1..1 0..*

Postulacion
- - id: UUID
- - usuario: Usuario
- - scrim: Scrim
- - rolDeseado: String
- - estado: EstadoPostulacion
- - fechaPostulacion: Date
- + Postulacion(usuario: Usuario, scrim: Scrim, rolDeseado: String)
- + aceptar(): void
- + rechazar(): void
- + getId(): UUID
- + getUsuario(): Usuario
- + getScrim(): Scrim
- + getRolDeseado(): String
- + setRolDeseado(rol: String): void
- + getEstado(): EstadoPostulacion
- + setEstado(estado: EstadoPostulacion): void
- + getFechaPostulacion(): Date

contiene 1..1 0..*
tiene estado 1..1 1..1

EstadoPostulacion
- - nombre: String
- + EstadoPostulacion(nombre: String)
- + getNombre(): String
- + esPendiente(): boolean
- + esAceptada(): boolean
- + esRechazada(): boolean

events

«Subject, Singleton» DomainEventBus
- - instance: DomainEventBus
- - subscribers: List<Subscriber>
- # DomainEventBus()
- + getInstance(): DomainEventBus
- + subscribe(subscriber: Subscriber): void
- + unsubscribe(subscriber: Subscriber): void
- + publish(event: DomainEvent): void
- + clearSubscribers(): void

1 publica 0..*
1 notifica a 0..*
gestiona 1..1

«abstract» DomainEvent
- - id: UUID
- - occurredOn: Date
- + DomainEvent()
- + getId(): UUID
- + getOccurredOn(): Date

«Observer» Subscriber
- + onEvent(event: DomainEvent): void
- + getSubscriberName(): String

ScrimStateChangedEvent
- - scrimId: UUID
- - estadoAnterior: String
- - estadoNuevo: String
- + ScrimStateChangedEvent(scrimId: UUID, estadoAnterior: String, estadoNuevo: String)
- + getScrimId(): UUID
- + getEstadoAnterior(): String
- + getEstadoNuevo(): String

PostulacionAceptadaEvent
- - scrimId: UUID
- - usuarioId: UUID
- + PostulacionAceptadaEvent(scrimId: UUID, usuarioId: UUID)
- + getScrimId(): UUID
- + getUsuarioId(): UUID

administra

application

controller

«Controller» AuthController
- - scrimService: ScrimService
- + AuthController(scrimService: ScrimService)
- + registrarUsuario(username: String, email: String, password: String, region: String): Usuario
- + autenticar(email: String, password: String): Map<String, Object>

«Controller» ScrimController
- - scrimService: ScrimService
- + ScrimController(scrimService: ScrimService)
- + buscarScrims(filtros: Map<String, Object>): List<Scrim>
- + crearScrim(creadorId: UUID, datos: Map<String, Object>): Scrim
- + postularseAScrim(scrimId: UUID, usuarioId: UUID, rol: String): void
- + confirmarParticipacion(scrimId: UUID, usuarioId: UUID): void
- + iniciarScrim(scrimId: UUID): void
- + finalizarScrim(scrimId: UUID): void
- + cancelarScrim(scrimId: UUID): void
- + cargarEstadisticas(scrimId: UUID, datos: Map<String, Object>): Map<String, Object>
- + ejecutarComando(scrimId: UUID, comando: String): Map<String, Object>

delega autenticación en 1..1 1..1
delega operaciones en 1..1 1..1

service

«Service» ScrimService
- - scrimRepository: Map<UUID, Scrim>
- - usuarioRepository: Map<UUID, Usuario>
- - matchmakingStrategy: MatchmakingStrategy
- + ScrimService(strategy: MatchmakingStrategy)
- + crearScrim(creador: Usuario, juego: String, formato: String, region: String): Scrim
- + postularseAScrim(scrimId: UUID, usuarioId: UUID, rolDeseado: String): void
- + emparejarYArmarLobby(scrimId: UUID): void
- + confirmarParticipacion(scrimId: UUID, usuarioId: UUID): void
- + iniciarScrim(scrimId: UUID): void
- + finalizarScrim(scrimId: UUID): void
- + cancelarScrim(scrimId: UUID): void
- + procesarReporte(reporte: ReporteConducta): void
- + setMatchmakingStrategy(strategy: MatchmakingStrategy): void
- + getScrimRepository(): Map<UUID, Scrim>
- + getUsuarioRepository(): Map<UUID, Usuario>

1 1 1 1

builder

«Builder» ScrimBuilder
- - scrim: Scrim
- + ScrimBuilder(creador: Usuario, juego: String, formato: String, region: String)
- + conRangos(rangoMin: String, rangoMax: String): ScrimBuilder
- + conLatenciaMaxima(latencia: int): ScrimBuilder
- + conFechaHora(fechaHora: LocalDateTime): ScrimBuilder
- + conCupos(cupos: int): ScrimBuilder
- + conModalidad(modalidad: String): ScrimBuilder
- + build(): Scrim

construye 1 1

facade

«Facade» EscrimsFacade
- - scrimService: ScrimService
- - eventBus: DomainEventBus
- - notificationSubscriber: NotificationSubscriber
- - notifierFactory: NotifierFactory
- + EscrimsFacade(strategy: MatchmakingStrategy)
- + crearUsuario(username: String, email: String, password: String, region: String): Usuario
- + crearScrim(creadorId: UUID, juego: String, formato: String, region: String): Scrim
- + buscarScrims(): List<Scrim>
- + postularseAScrim(scrimId: UUID, usuarioId: UUID, rol: String): void
- + confirmarParticipacion(scrimId: UUID, usuarioId: UUID): void
- + iniciarScrim(scrimId: UUID): void
- + finalizarScrim(scrimId: UUID): void
- + cancelarScrim(scrimId: UUID): void
- + cambiarEstrategiaMatchmaking(strategy: MatchmakingStrategy): void
- + asignarRol(scrimId: UUID, usuarioId: UUID, rol: String): void
- + intercambiarJugadores(scrimId: UUID, usuario1Id: UUID, usuario2Id: UUID): void
- + invitarJugador(scrimId: UUID, usuarioId: UUID, rol: String): void
- + reportarConductaInapropiada(scrimId: UUID, reportadorId: UUID, reportadoId: UUID, motivo: String): void

coordina casos de uso con 1..1 1..1
construye scrims con 1..1 1..1

subscribers

«Observer» NotificationSubscriber
- - notifierFactory: NotifierFactory
- - scrimRepository: Map<UUID, Scrim>
- - usuarioRepository: Map<UUID, Usuario>
- + NotificationSubscriber(factory: NotifierFactory)
- + onEvent(event: DomainEvent): void
- + getSubscriberName(): String
- + registrarUsuario(usuario: Usuario): void
- + registrarScrim(scrim: Scrim): void

publica eventos mediante 1..1 1..1
crea notificadores con 1..1 1..1
registra notificaciones en 1..1 1..1

infrastructure

notifications

«Product» Notifier
- + send(destinatario: Usuario, titulo: String, mensaje: String): void
- + getChannelName(): String

«AbstractFactory» NotifierFactory
- + createPushNotifier(): Notifier
- + createEmailNotifier(): Notifier
- + createDiscordNotifier(): Notifier

«ConcreteFactory» DevNotifierFactory
- + DevNotifierFactory()
- + createPushNotifier(): Notifier
- + createEmailNotifier(): Notifier
- + createDiscordNotifier(): Notifier

«ConcreteFactory» ProdNotifierFactory
- + ProdNotifierFactory()
- + createPushNotifier(): Notifier
- + createEmailNotifier(): Notifier
- + createDiscordNotifier(): Notifier

«ConcreteProduct» EmailNotifier
- + EmailNotifier()
- + send(destinatario: Usuario, titulo: String, mensaje: String): void
- + getChannelName(): String

«ConcreteProduct» PushNotifier
- + PushNotifier()
- + send(destinatario: Usuario, titulo: String, mensaje: String): void
- + getChannelName(): String

«ConcreteProduct» DiscordNotifier
- + DiscordNotifier()
- + send(destinatario: Usuario, titulo: String, mensaje: String): void
- + getChannelName(): String

crea 1..1 1..1

adapters

«Adapter» ExternalServiceAdapter
- + estaDisponible(): boolean
- + getNombre(): String

«ConcreteAdapter» ICalAdapter
- - conectado: boolean
- + ICalAdapter()
- + conectar(): void
- + desconectar(): void
- + exportarEvento(titulo: String, fecha: LocalDateTime, duracion: int): String
- + estaDisponible(): boolean
- + getNombre(): String

«ConcreteAdapter» SendGridAdapter
- - apiKey: String
- - conectado: boolean
- + SendGridAdapter(apiKey: String)
- + conectar(): void
- + desconectar(): void
- + enviarEmail(destinatario: String, asunto: String, contenido: String): void
- + estaDisponible(): boolean
- + getNombre(): String

«ConcreteAdapter» DiscordAdapter
- - webhookUrl: String
- - conectado: boolean
- + DiscordAdapter(webhookUrl: String)
- + conectar(): void
- + desconectar(): void
- + enviarMensaje(canal: String, mensaje: String): void
- + estaDisponible(): boolean
- + getNombre(): String

selecciona candidatos con
procesa reportes mediante
ejecuta comandos con
consulta scrims
consulta usuarios
envía notificaciones con 1..1 0..*
solicita notificadores a

Referencias
«Controller» = Controlador
«Service» = Servicio de aplicación
«Facade» = Fachada
«Builder» = Builder
«State» = Patrón State
«Strategy» = Patrón Strategy
«Command» = Patrón Command
«Observer» = Patrón Observer
«Subject» = Sujeto observable
«AbstractFactory» = Abstract Factory
«Adapter» = Adapter
«ChainOfResponsibility» = Chain of Responsibility
«Singleton» = Instancia única